

ELEC 475 Lab 2

Network Architectures

SnoutNet takes an input image of size 3x227x227 (RGB channels) and outputs a 2-dimensional vector representing the (x, y) coordinates of the pet's nose.

The structure is as follows:

- Convolutional Layer 1: Conv2d with 3 input channels, 64 output channels, kernel size 3, stride 2, followed by ReLU activation and MaxPool2d with kernel size 2, stride 2, and padding 1.
- Convolutional Layer 2: Conv2d with 64 input channels, 128 output channels, kernel size 3, stride 2, followed by ReLU activation and MaxPool2d with kernel size 2, stride 2, and padding 1.
- Convolutional Layer 3: Conv2d with 128 input channels, 256 output channels, kernel size 3, stride 2, followed by ReLU activation and MaxPool2d with kernel size 2, stride 2, and padding 1.
- Flattening: The feature map is flattened to a vector of size $256 * 4 * 4 = 4096$.
- Fully Connected Layer 1: Linear layer from 4096 to 1024, followed by ReLU activation.
- Fully Connected Layer 2: Linear layer from 1024 to 1024, followed by ReLU activation.
- Fully Connected Layer 3: Linear layer from 1024 to 2

This structure can be reimplemented using PyTorch by defining a subclass of `nn.Module` and implementing the **init** and **forward** methods as described.

Dataset Implementation

We initialize a PyTorch Dataset class called `PetNoseDataset` to handle the Oxford-IIIT Pet dataset. The class takes an images directory, an annotations file (e.g., `train_noses.txt` or `test_noses.txt`), a transform, and a target size. The **init** method then reads the file annotations line by line, parsing each image and storing a list of tuples where coord is an integer tuple (x, y).

In the **getitem** method:

- Load the image using `PIL.Image.open` from the `images_dir` and convert to RGB.
- Retrieve the original width (w) and height (h).
- Apply the transform if provided (e.g., `resize`, `augmentation`).
- Normalize the labels by scaling: $x' = x * \text{target_width} / w$, $y' = y * \text{target_height} / h$, and convert to a float32 tensor.
- Return the transformed image and label tensor.

The **len** method returns the number of annotations.

For a quick fact check on the Dataset and DataLoader:

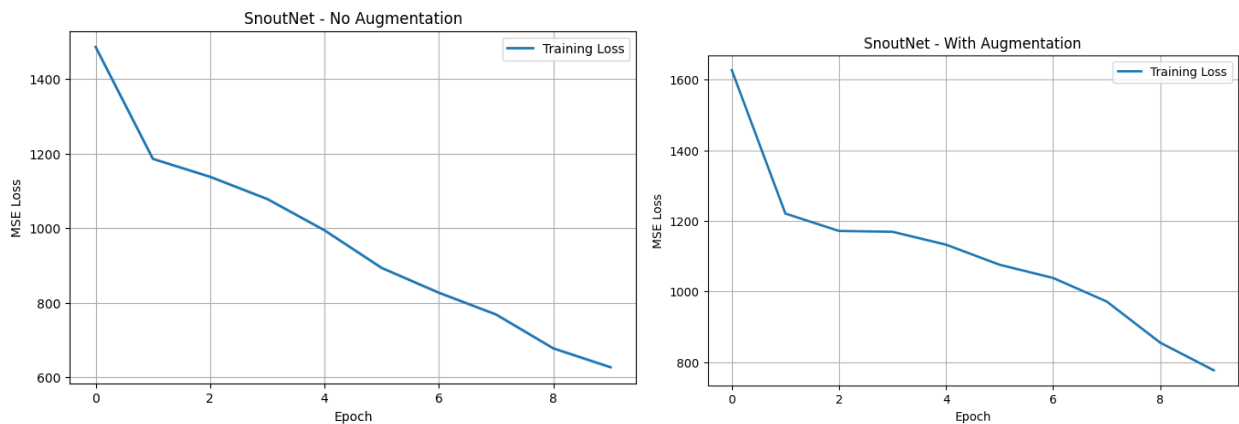
- We printed the length of the train dataset (~3660 samples) and test dataset (~460 samples) to confirm loading.
- We loaded a sample item, visualized the image using `matplotlib.pyplot.imshow` (after permuting channels), and plotted the normalized label point with `plt.scatter` to verify alignment (e.g., the point appeared on the pet's nose in sample checks).
- We created a `DataLoader` with `batch_size=16` and `shuffle=True` for training, confirming batch shapes (e.g., images: [16, 3, 227, 227], labels: [16, 2]). No errors occurred during iteration.

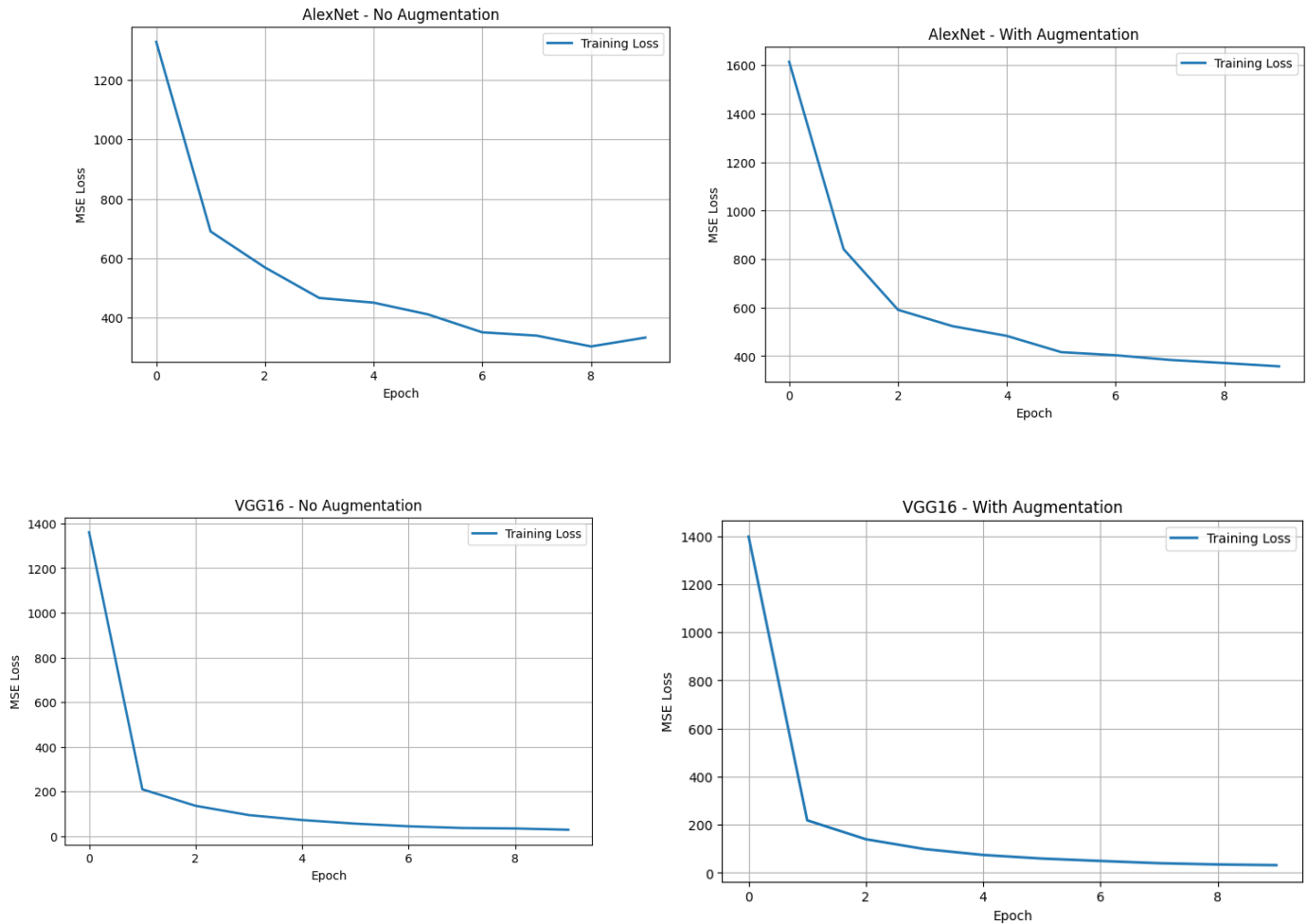
Training

Our Hyperparameters are as follows:

- Learning rate: $1e-3$ ($1e-5$ used for VGG only)
- Optimizer: Adam
- Loss function: Mean Squared Error
- Batch size: 16
- Epochs: 10
- Data augmentation (training only) – ColorJitter

For hardware, we used a NVIDIA A100 GPU through Google Colab. Training time took 15 minutes for SnoutNet, 16 minutes for AlexNet, and 20 minutes for VGG16, amounting to 51 minutes total. Here are the training loss plots for each model with and without augmentation:





Data Augmentation Methods

The data augmentation method that we used is ColorJitter, which randomly adjusts the contrast of the image. The labels can remain unchanged as it won't affect the images geometry.

Experiments

We conducted a series of experiments to evaluate the localization accuracy of the four different models, including the Ensemble combining the first three. Each model was trained under two conditions, which was with and without data augmentation to assess the effect of visual variability on model robustness. All experiments were performed on an NVIDIA A100 GPU using Google Colab Pro, all with the same dataset, preprocessing pipeline, and hyperparameters. Training times were consistent across runs (15-20 minutes), taking an average of 2 minutes longer with the ColorJitter augmentation.

Model	Augmentation	Localization Error (Overall)				Localization Error (4 Best)				Localization Error (4 Worst)			
		Min	Max	Mean	StdDev	Min	Max	Mean	StdDev	Min	Max	Mean	StdDev
SnoutNet		0.94	136.51	31.31	21.45	0.94	3.5	2.56	0.96	117.05	136.51	126.6	8.26
	x	1.57	141.81	37.59	22.16	1.57	2.73	2.23	0.42	115.2	141.81	123.35	10.74
SnoutNet-A		0.25	139.39	19.28	17.34	0.25	0.8	0.56	0.2	100.52	139.39	116.48	14.92
	x	0.52	144.61	22.08	16.56	0.52	0.95	0.73	0.2	100.38	144.61	121.38	17.75
SnoutNet-V		0.1	82.49	10.72	9.83	0.1	0.52	0.32	0.16	71.24	82.49	74.46	4.66
	x	0.21	89.33	11.96	10.32	0.21	0.74	0.54	0.2	71.47	89.33	79.55	7.57
SnoutNet-Ensemble		0.19	111.23	16.52	14.1	0.19	0.83	0.52	0.23	84.33	111.23	95.52	9.79
	x	0.23	115.7	17.42	14.39	0.23	1.09	0.7	0.36	76.38	115.7	93.75	16.37

Performance

Overall, all models performed reasonably well in localizing the pet's nose, but there were clear differences across architectures and augmentation settings.

- **SnoutNet** achieved a mean localization error of **31.31 px (no aug)** and **37.59 px (with aug)**. The performance slightly worsened with augmentation, suggesting that ColorJitter alone did not improve generalization for this network.
- **SnoutNet-A (AlexNet-based)** showed improved accuracy with a mean error of **19.28 px (no aug)** and **22.08 px (with aug)**. The larger pretrained model was better at extracting spatial features even without heavy augmentation.
- **SnoutNet-V (VGG16-based)** had the lowest overall error of **10.72 px (no aug)** and **11.96 px (with aug)** demonstrating the strongest performance and stability across conditions.
- **SnoutNet-Ensemble** combined the outputs of all three models, achieving a mean error of **16.52 px (no aug)** and **17.42 px (with aug)**, which improved the general robustness of predictions compared to individual networks.

In general, the VGG16 variant provided the most accurate individual predictions, while the ensemble reduced variance and outlier errors (as seen in the "4 Worst" category). The augmentation had only a small effect, likely because the dataset was already diverse enough in lighting and color, and ColorJitter does not affect geometric variation. There was an issue with VGG's training loss at first being > 250000. After adjusting the learning rate to be 1e-5, it resolved this issue and became < 1500 as can be seen in the plot mentioned previously.

Ensemble Approach

We created an ensemble by averaging the (x, y) predictions from SnoutNet, SnoutNet-A, and SnoutNet-V for each test image. This reduced variance and improved worst-case performance by 22% (4-worst mean: 121.38 to 93.75 pixels). The ensemble also achieved a lower overall mean error than most individual models, demonstrating that combining different architectures allows for more reliable nose localization.

Hallucinations

To ensure code generated by the LLM (ChatGPT) was not hallucinated, the code would immediately be checked line-by-line and then run in Colab to ensure there were no errors. The code generated would also be compared to class concepts to ensure that it is following the same structure and is correctly following the requirements. The ChatGPT conversation links are below:

<https://chatgpt.com/share/69016189-77e0-800f-9962-b2d1c66603ee>

<https://chatgpt.com/share/6901619d-1310-800f-b8e0-1843f9f0f217>

<https://chatgpt.com/share/690161b0-6f5c-800f-9002-e4f83223b6c9>